

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»

Отчет по лабораторной работе №3-4
«Функциональные возможности языка Python»

Выполнил:
Студент группы ИУ5-31Б
Блинкова Ксения

Проверил:
Гапанюк Ю.Е.

2025 г.

Задание

Задание лабораторной работы состоит из решения нескольких задач. Файлы, содержащие решения отдельных задач, должны располагаться в пакете `lab_python_fr`. Решение каждой задачи должно располагаться в отдельном файле.

При запуске каждого файла выдаются тестовые результаты выполнения соответствующего задания.

Задача 1 (файл `field.py`)

Необходимо реализовать генератор `field`. Генератор `field` последовательно выдает значения ключей словаря. Пример:

- В качестве первого аргумента генератор принимает список словарей, дальше через `*args` генератор принимает неограниченное количество аргументов.
- Если передан один аргумент, генератор последовательно выдает только значения полей, если значение поля равно `None`, то элемент пропускается.
- Если передано несколько аргументов, то последовательно выдаются словари, содержащие данные элементы. Если поле равно `None`, то оно пропускается. Если все поля содержат значения `None`, то пропускается элемент целиком.

Задача 2 (файл `gen_random.py`)

Необходимо реализовать генератор `gen_random(количество, минимум, максимум)`, который последовательно выдает заданное количество случайных чисел в заданном диапазоне от минимума до максимума, включая границы диапазона. Пример:

`gen_random(5, 1, 3)` должен выдать 5 случайных чисел в диапазоне от 1 до 3, например 2, 2, 3, 2, 1

Задача 3 (файл `unique.py`)

- Необходимо реализовать итератор `Unique(данные)`, который принимает на вход массив или генератор и итерируется по элементам, пропуская дубликаты.
- Конструктор итератора также принимает на вход именованный `bool`-параметр `ignore_case`, в зависимости от значения которого будут считаться одинаковыми строки в разном регистре. По умолчанию этот параметр равен `False`.
- При реализации необходимо использовать конструкцию `**kwargs`.
- Итератор должен поддерживать работу как со списками, так и с генераторами.
- Итератор не должен модифицировать возвращаемые значения.

Задача 4 (файл sort.py)

Дан массив 1, содержащий положительные и отрицательные числа.

Необходимо **одной строкой кода** вывести на экран массив 2, который содержит значения массива 1, отсортированные по модулю в порядке убывания. Сортировку необходимо осуществлять с помощью функции `sorted`.

Пример:

```
data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
```

```
Вывод: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
```

Необходимо решить задачу двумя способами:

1. С использованием `lambda`-функции.
2. Без использования `lambda`-функции.

Задача 5 (файл print_result.py)

Необходимо реализовать декоратор `print_result`, который выводит на экран результат выполнения функции.

- Декоратор должен принимать на вход функцию, вызывать её, печатать в консоль имя функции и результат выполнения, после чего возвращать результат выполнения.
- Если функция вернула список (`list`), то значения элементов списка должны выводиться в столбик.
- Если функция вернула словарь (`dict`), то ключи и значения должны выводиться в столбик через знак равенства.

Задача 6 (файл cm_timer.py)

Необходимо написать контекстные менеджеры `cm_timer_1` и `cm_timer_2`, которые считают время работы блока кода и выводят его на экран. Пример:

```
with cm_timer_1():
```

```
    sleep(5.5)
```

После завершения блока кода в консоль должно вывестись `time:`

5.5 (реальное время может несколько отличаться).

`cm_timer_1` и `cm_timer_2` реализуют одинаковую функциональность, но должны быть реализованы двумя различными способами (на основе класса и с использованием библиотеки `contextlib`).

Листинг кода

field.py

```
def field(items, *args):
    assert len(args) > 0

    if len(args) == 1:
        key = args[0]
        for item in items:
            if key in item and item[key] is not None:
                yield item[key]
    else:
        for item in items:
            result = {}
```

```

        has_valid_fields = False
    for key in args:
        if key in item and item[key] is not None:
            result[key] = item[key]
            has_valid_fields = True
    if has_valid_fields:
        yield result

```

gen_random.py

```

import random

def gen_random(num_count, begin, end):
    for i in range(num_count):
        yield random.randint(begin, end)

```

unique.py

```

class Unique(object):
    def __init__(self, items, **kwargs):
        self.ignore_case = kwargs.get('ignore_case', False)
        self.items = iter(items)
        self.seen = set()

    def __next__(self):
        while True:
            item = next(self.items)

            if self.ignore_case and isinstance(item, str):
                key = item.lower()
            else:
                key = item

            if key not in self.seen:
                self.seen.add(key)
                return item

    def __iter__(self):
        return self

```

sort.py

```

def sort_without_lambda(data):
    return sorted(data, key=abs, reverse=True)

def sort_with_lambda(data):
    return sorted(data, key=lambda x: abs(x), reverse=True)

data = [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]

```

print_result.py

```
def print_result(func):
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        print(func.__name__)

        if isinstance(result, list):
            for item in result:
                print(item)
        elif isinstance(result, dict):
            for key, value in result.items():
                print(f"{key} = {value}")
        else:
            print(result)

    return result
return wrapper
```

cm_timer.py

```
import time
from contextlib import contextmanager

class cm_timer_1:
    def __enter__(self):
        self.start_time = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        elapsed_time = time.time() - self.start_time
        print(f"time: {elapsed_time:.1f}")

@contextmanager
def cm_timer_2():
    start_time = time.time()
    try:
        yield
    finally:
        elapsed_time = time.time() - start_time
        print(f"time: {elapsed_time:.1f}")
```

main.py

```
from field.field import field
from gen_random.gen_random import gen_random
from unique.unique import Unique
from sort.sort import data as sort_data
from print_result.print_result import print_result
from cm_timer.cm_timer import cm_timer_1, cm_timer_2
import time

def main():
```

```

print("=== Лабораторная работа: Функциональное программирование ===\n")

# Тест 1: Генератор field
print("1. Тест field - работа со словарями:")
goods = [
    {'title': 'Ковер', 'price': 2000, 'color': 'green'},
    {'title': 'Диван для отдыха', 'color': 'black'},
    {'title': None, 'price': 3000},
    {'price': 2500, 'color': 'white'},
    {'title': 'Стул', 'price': None, 'color': 'brown'}
]

print("    • Один аргумент 'title':")
for title in field(goods, 'title'):
    print(f"        {title}")

print("\n    • Несколько аргументов 'title', 'price':")
for item in field(goods, 'title', 'price'):
    print(f"        {item}")

# Тест 2: Генератор случайных чисел
print("\n2. Тест gen_random - генератор случайных чисел:")
print("    • 5 чисел от 1 до 3:", list(gen_random(5, 1, 3)))
print("    • 10 чисел от 0 до 100:", list(gen_random(10, 0, 100)))

# Тест 3: Итератор Unique - удаление дубликатов
print("\n3. Тест Unique - удаление дубликатов:")

print("    • Числа:", list(Unique([1, 1, 1, 2, 2, 2])))

strings_data = ['a', 'A', 'b', 'B', 'a', 'A', 'b', 'B']
print("    • Строки (разный регистр):", list(Unique(strings_data)))
print("    • Строки (игнорировать регистр):", list(Unique(strings_data,
ignore_case=True)))

print("    • С генератором:", list(Unique(gen_random(10, 1, 3))))

# Тест 4: Сортировка по модулю
print("\n4. Тест сортировки по модулю:")
print("    • Исходные данные:", sort_data)

result1 = sorted(sort_data, key=abs, reverse=True)
result2 = sorted(sort_data, key=lambda x: abs(x), reverse=True)

print("    • Без lambda:", result1)
print("    • С lambda:", result2)
print("    • Результаты совпадают:", result1 == result2)

# Тест 5: Декоратор print_result
print("\n5. Тест print_result - декоратор для вывода результатов:")

@print_result
def simple_number():
    return 42

```

```
@print_result
def simple_string():
    return "Hello BMSTU"

@print_result
def simple_dict():
    return {'name': 'Ivan', 'age': 20}

@print_result
def simple_list():
    return [1, 2, 3, 4, 5]

simple_number()
simple_string()
simple_dict()
simple_list()

# Тест 6: Контекстные менеджеры для замера времени
print("\n6. Тест cm_timer - контекстные менеджеры:")

print("    • cm_timer_1 (на основе класса):")
with cm_timer_1():
    time.sleep(1.2)

print("    • cm_timer_2 (с использованием contextlib):")
with cm_timer_2():
    time.sleep(0.8)

print("\n=== Все тесты завершены! ===")

if __name__ == '__main__':
    main()
```

Результат выполнения

```
ksu@LAPTOP-C4SGKB4C:~/Blinkova_Labs_2025/lab_python_fp$ python3 main.py
=== Лабораторная работа: Функциональное программирование ===
```

1. Тест field - работа со словарями:

- Один аргумент 'title':
Ковер
Диван для отдыха
Стул
- Несколько аргументов 'title', 'price':
{'title': 'Ковер', 'price': 2000}
{'title': 'Диван для отдыха'}
{'price': 3000}
{'price': 2500}
{'title': 'Стул'}

2. Тест gen_random - генератор случайных чисел:

- 5 чисел от 1 до 3: [3, 3, 3, 3, 2]
- 10 чисел от 0 до 100: [69, 71, 26, 69, 26, 87, 0, 8, 31, 63]

3. Тест Unique - удаление дубликатов:

- Числа: [1, 2]
- Строки (разный регистр): ['a', 'A', 'b', 'B']
- Строки (игнорировать регистр): ['a', 'b']
- С генератором: [1, 3, 2]

4. Тест сортировки по модулю:

- Исходные данные: [4, -30, 30, 100, -100, 123, 1, 0, -1, -4]
- Без lambda: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
- С lambda: [123, 100, -100, -30, 30, 4, -4, 1, -1, 0]
- Результаты совпадают: True

5. Тест print_result - декоратор для вывода результатов:

```
simple_number
42
simple_string
Hello BMSTU
simple_dict
name = Ivan
age = 20
simple_list
1
2
3
4
5
```

6. Тест cm_timer - контекстные менеджеры:

- cm_timer_1 (на основе класса):
time: 1.2
- cm_timer_2 (с использованием contextlib):
time: 0.8

```
=== Все тесты завершены! ===
```


